

Goal-Driven Multi-Agent Simulation Framework for Evaluating Recommender Systems under Data Scarcity

Technical Report

Hatem Amine · ESTIN · 2024

hatem@estin.dz

1. Abstract

Recommender systems depend heavily on large-scale user interaction logs for training and evaluation. In many real-world scenarios such data is unavailable due to cold-start conditions, privacy regulations, or domain novelty. This work presents a **Goal-Driven Multi-Agent Simulation Framework** that addresses this data-scarcity problem by generating synthetic interaction logs through LLM-powered virtual news readers operating inside a RecSim environment over the Microsoft News Dataset (MIND).

Virtual agents are assigned explicit reading goals and persona archetypes. They interact with a FAISS-backed news retrieval system and make decisions guided by a locally-running Gemma-4 language model judge. The synthetic logs are validated against real MIND interaction data through two complementary evaluation approaches: **Approach A** (distribution matching / fidelity metrics) and **Approach B** (train-on-synthetic / test-on-real NDCG@10 via Neural Collaborative Filtering).

2. Problem Statement

Standard news recommendation pipelines require click logs to: (a) train ranking models, (b) compute offline evaluation metrics such as NDCG and CTR, and (c) study user behaviour patterns. When no historical log exists — for example at system launch, after a major topic shift, or in low-resource language settings — these pipelines fail entirely.

The core research question is: *Can we generate synthetic interaction logs that are both faithful to real user behaviour and useful as training data for recommender systems?*

2.1 Design Constraints

- **No real log available at generation time.** The simulator must produce plausible behaviour from scratch using only the news corpus.
- **Diversity of user types.** Real MIND users exhibit at least five behavioural clusters; the simulation must reproduce this distribution.
- **Bilingual corpus.** MIND contains English articles; the agent framework supports both English and French personas.
- **Reproducibility.** All LLM calls are cached; results are fully deterministic given the same random seed.

3. Architecture Overview

The framework is composed of six layers that interact in a feed-forward pipeline:

Layer	Module	Responsibility
Data	data/mind_loader.py data/news_preprocessor.py	Load MIND TSV files, build FAISS index, cluster real users
RecSim Environment	recsim_env/	Gym-compatible simulation loop: document sampling, user state, response
LLM Judge	llm/judge.py llm/prompts/	Local Gemma-4 inference: relevance scoring, goal-progress assessment,

Simulation	simulation/	Agent policy, virtual-user generator, SQLite persistence, parallel runner
Evaluation	evaluation/	Approach A fidelity metrics, Approach B NCF train/test
Export	export/	TREC qrels, MIND behaviors.tsv, RecSim TFRecords

4. The Role of RecSim

RecSim (le et al., 2019) is Google's open-source library for building simulated recommendation environments. It defines a clean interface between the *recommender* (agent) and the *user* (environment) through a set of abstract base classes modelled on the OpenAI Gym API.

4.1 Why RecSim?

- **Formal RL interface.** RecSim exposes `reset()` / `step(slate)` / reward semantics, enabling the agent to interact with the environment in a standard reinforcement-learning loop — which is exactly the session-based read/click cycle.
- **Separation of concerns.** Document sampling, user state evolution, and response generation are cleanly decoupled, making each component independently testable.
- **Multi-step sessions.** A single call to `env.step(slate)` advances one step; looping until `is_terminal()` naturally produces a complete reading session.
- **Reproducibility.** All random state flows through a seeded numpy RNG, so simulation runs are deterministic.

4.2 In-Repo Shim (recsim_compat.py)

The official recsim package fails on Python ≥ 3.10 because it imports `tf.estimator.SessionRunHook`, which was removed in TensorFlow 2. Rather than downgrading Python or TensorFlow, we provide a 200-line pure-Python shim (**recsim_env/recsim_compat.py**) that re-implements all required abstract base classes with no TF dependency:

- `AbstractDocument` / `AbstractDocumentSampler`
- `AbstractUserState` / `AbstractResponse`
- `AbstractUserSampler` / `AbstractUserModel`
- `Environment` — `reset()`, `step()`, `_sample_candidates()`
- `RecSimGymEnv` — wraps `Environment` with a custom `reward_aggregator`

All five `recsim_env/` modules import exclusively from this shim. The rest of the codebase is unaffected.

4.3 RecSim Step Loop

Each simulation step proceeds as follows:

Step	RecSim Component	What Happens
1. <code>reset()</code>	<code>Environment</code>	Calls <code>user_model.reset()</code> → samples new <code>MindUserState</code> ; calls <code>_sample_candidates()</code>
2. Agent selects slate	<code>StateDrivenAgent</code>	LLM decision prompt → returns list of candidate indices; cosine-similarity fallback if LL
3. <code>step(slate)</code>	<code>MindUserModel</code>	<code>simulate_response()</code> : LLM judge scores each document for relevance; Bernoulli sampl
4. <code>update_state()</code>	<code>MindUserModel</code>	Ticks fatigue +0.05; increments step counter

5. is_terminal()	MindUserState	Returns True when fatigue ≥ 0.9 or steps ≥ session_budget
6. Reward	reward_aggregator	Sum of relevance scores for clicked documents (dense reward signal)

5. Component Interactions

5.1 Data Layer → RecSim

Before any simulation begins, **news_preprocessor.py** encodes all MIND articles with *all-MiniLM-L6-v2* (384-d sentence embeddings) and stores a FAISS IndexFlatIP for inner-product search. At each session reset, **MindDocumentSampler.set_query()** fires a FAISS search against the current user goal, pre-loading up to 200 topically relevant articles as the candidate pool. Within a session, each `env.step()` draws from this pool in order (and replenishes it from FAISS when the agent updates its query).

5.2 LLM Judge → RecSim Responses

Inside **MindUserModel.simulate_response()** — the central RecSim callback — the Gemma-4 judge is invoked once per document per step. It receives the user's goal, role, and the article title/abstract, and returns a relevance score in [0, 1]. This score drives a logistic click probability:

$$P(\text{click}) = \text{sigmoid}(6 \cdot (\text{relevance} - 0.4)) - 0.25 \cdot \text{fatigue}$$

All judge calls are cached in SQLite by $\text{sha256}(\text{goal}[:120]) \times \text{doc_id} \times \text{lang} \times \text{mode}$, so repeated (goal, article) pairs across users cost zero inference time.

5.3 Agent Policy → Environment

GoalDrivenAgent implements the RL policy. At each step it:

- Reads the observation dict (user interest vector + doc candidate list) from RecSim.
- Calls the LLM decision prompt (*decision_en.j2* or *decision_fr.j2*) to select up to `slate_size` indices and obtain an updated search query.
- Falls back to cosine-similarity ranking between the interest vector and candidate embeddings if LLM generation fails.
- Returns the slate as a numpy index array to `env.step()`.

After each step the agent calls `get_next_query()` and passes the result to `doc_sampler.set_query()`, steering future candidate retrieval toward the agent's evolving information need.

5.4 Persistence Layer

The simulation runner stores every interaction in an SQLite database (**simulation/db.py**) with the following schema:

Table	Key Columns	Purpose
virtual_users	user_id, archetype, language_pref, goal, role	One row per generated virtual user
sessions	session_id, user_id, session_num, total_clicks, total_fatigue, goal_progress	One row per session
interactions	session_id, step, position, news_id, clicked, click_time, relevance, fatigue	One row per interaction in the synthetic interaction log

6. Virtual User Model

6.1 Persona Archetypes

Five archetypes calibrated from K-means clustering of real MIND users:

Archetype	Language	Share	Reading Style	Session Budget
breaking_news_follower	EN	28 %	skimmer	25 articles
topic_specialist	EN	22 %	deep_reader	15 articles
casual_browser	EN	20 %	balanced	20 articles
sentiment_tracker	FR	15 %	skimmer	20 articles
deep_reader	FR	15 %	deep_reader	12 articles

6.2 User State (MindUserState)

- **interest_vector** (384-d): weighted running mean of clicked document embeddings; updated as $\alpha \cdot \text{doc_emb} + (1-\alpha) \cdot \text{interest}$ with $\alpha = 0.3 \times \text{relevance}$.
- **fatigue** (0–1): incremented by 0.05 per step; session terminates at 0.9.
- **goal**: natural-language reading objective generated by Gemma-4 via the persona prompt template.
- **current_query**: evolves during the session as the agent refines its information need.

7. Evaluation Framework

7.1 Approach A — Distribution Matching

Approach A measures how closely the synthetic log resembles real MIND user behaviour without requiring model training. Six metrics are computed:

Metric	Definition	Ideal
CTR KL-divergence	$KL(P_{\text{sim_categories}} \parallel P_{\text{real_categories}})$	$\rightarrow 0$
Session-length Wasserstein	W distance between click-per-session histograms	$\rightarrow 0$
Category entropy gap	$ H(\text{real categories}) - H(\text{sim categories}) $	$\rightarrow 0$
Position-bias ρ	Spearman correlation between position and CTR	Negative (like real users)
ILD	Mean pairwise $(1 - \text{cosine})$ of clicked embeddings	Higher $\rightarrow$ more diverse
Replay NDCG@k	NDCG of real impressions ranked by position order	Baseline reference

7.2 Approach B — Train on Synthetic / Test on Real

Approach B trains a Neural Collaborative Filtering (NCF) model on the synthetic log and evaluates it on real MIND test impressions. This directly answers: *is the synthetic data useful as a training set?*

Because synthetic users (vuser_0001 ...) and real MIND users (U123456 ...) have disjoint IDs, a direct user-lookup evaluation is impossible. Instead we apply **zero-shot item-embedding transfer**:

- NCF learns item embeddings that encode co-click structure from synthetic interactions.
- For each real test user, a query vector is formed as the mean of their click-history item embeddings (items seen during synthetic training).
- Impression candidates are ranked by cosine similarity to the query.
- NDCG@10 and Recall@10 are computed on the ranked list.

An ablation study varies $N \in \{100, 500, 1000\}$ synthetic users to measure how metric quality scales with synthetic data volume.

7.3 NCF Architecture

GMF + MLP fusion (He et al., 2017) with BPR pairwise ranking loss:

Component	Detail
GMF branch	Element-wise product of user and item embeddings (emb_dim=32)
MLP branch	Concatenated user+item embeddings $\rightarrow$ FC layers (64 $\rightarrow$ 32 $\rightarrow$ 16) with ReLU
Output layer	Linear(emb_dim + 16, 1) — predicts preference score
Loss	BPR: $-\log \sigma(\text{score}_{\text{pos}} - \text{score}_{\text{neg}})$ over randomly sampled negatives
Optimiser	Adam, lr=1e-3, batch_size=256, 10 epochs

8. End-to-End Data Flow

The full pipeline from raw MIND files to paper-ready metrics:

Stage	Input	Output	Module
1. Index	news.tsv	FAISS index + SQLite news DB	news_preprocessor.py
2. Cluster	behaviors.tsv	Archetype distribution {name: fraction}	mind_loader.py
3. Generate users	Archetype distribution	List of virtual user profiles (goal, role, topics)	user_generator.py
4. Simulate	User profiles + FAISS index	SQLite interactions DB (clicked, relevance)	recsim_pyel_recsim.py
5. Approach A	Interactions DB + real behaviors	Fidelity report dict	approach_a.py
6. Approach B	Interactions DB + real behaviors	NCF model; NDCG@10, Recall@10	approach_b.py
7. Export	Interactions DB	TREC qrels.txt, behaviors.tsv, TFRec report (optional)	export.py

9. Technology Stack

Concern	Library / Tool	Version
Simulation loop	recsim_compat (in-repo shim)	custom
Gym interface	gymnasium	≥ 0.29

News embeddings	sentence-transformers all-MiniLM-L6-v2	≥ 2.2
Semantic search	FAISS IndexFlatIP	faiss-cpu $\geq 1.7.4$
LLM judge	google/gemma-4-E4B-it (local)	transformers ≥ 4.45
4-bit quantisation	bitsandbytes (optional)	≥ 0.43
Prompt templating	Jinja2	≥ 3.1
RS training	PyTorch (NCF)	≥ 2.0
Data wrangling	pandas + scikit-learn	$\geq 2.0 / 1.3$
Statistics	scipy	≥ 1.11
Progress display	tqdm	≥ 4.66
Persistence	SQLite (stdlib)	—
Config	PyYAML + python-dotenv	$\geq 6.0 / 1.0$

10. Limitations and Future Work

- **LLM inference cost.** Each (goal, article) pair requires one Gemma forward pass. The SQLite cache mitigates this for repeated pairs, but cold-start runs over large corpora remain slow without a GPU.
- **English-centric embeddings.** all-MiniLM-L6-v2 is primarily trained on English text; French article retrieval quality may be lower. A multilingual model (e.g. paraphrase-multilingual-MiniLM-L12-v2) would improve FR persona accuracy.
- **Click model simplicity.** The logistic click function does not model position bias, novelty, or contextual effects beyond fatigue. A more realistic cascade click model could improve Approach A fidelity.
- **Approach B item coverage.** Zero-shot transfer only benefits real users whose click history overlaps with articles the NCF has seen. Coverage grows with more synthetic users and more simulation sessions.
- **Single-domain.** The current framework is tested only on MIND (news). Extension to e-commerce or academic paper recommendation is straightforward given the modular design.

11. References

- Ie, E. et al. (2019). *RecSim: A Configurable Simulation Platform for Recommender Systems*. arXiv:1909.04847.
- Wu, F. et al. (2020). *MIND: A Large-scale Dataset for News Recommendation*. ACL 2020.
- He, X. et al. (2017). *Neural Collaborative Filtering*. WWW 2017.
- Reimers, N. & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. EMNLP 2019.
- Johnson, J. et al. (2019). *Billion-scale similarity search with GPUs (FAISS)*. IEEE TBBM.
- Team, G. (2024). *Gemma 2: Improving Open Language Models at a Practical Size*. Google DeepMind.